

Proyecto 3 - VeriLang en Rascal

Documentacion final actualizada - version enfocada en rubrica 15/15

Curso: ISIS-2111 - Elementos Esenciales de Lenguajes de Programacion

Estudiante: Ronny Esteban Lopez Martinez

Codigo: 202415741

Proyecto Rascal: PROYECTO_RASCAL / rascaldslverilang

Entrega: ZIP del proyecto Rascal + este documento explicativo

Resumen ejecutivo

Este documento reemplaza la documentacion anterior y se enfoca especificamente en los tres criterios de la rubrica donde era necesario reforzar la entrega: **Grammar Update & Type Annotations, Type Correspondence Checking y Data Structure Element Existence Rule**. La version actual de VeriLang agrega estructuras de datos tipadas por el usuario, propaga correctamente el tipo del elemento dentro de cuantificadores y valida que todo tipo de usuario utilizado exista como defspace. El flujo completo parsea archivos .vl, ejecuta TypePal, bloquea la generacion ante errores semanticos y muestra el programa generado en consola cuando el chequeo es exitoso.

Criterio de rubrica	Estado actual	Evidencia clave
Grammar Update & Type Annotations	Nivel Excellent	La gramatica acepta defspace Group : Person end y Domain soporta int, bool, real, string, char y tipos definidos por usuario.
Type Correspondence Checking	Nivel Excellent	El checker valida firmas de operadores, relaciones, logica, reglas, cuantificadores y tipos dentro de estructuras tipadas.
Data Structure Element Existence Rule	Nivel Excellent	Si una estructura usa un tipo inexistente, por ejemplo defspace Group : Person end sin defspace Person end, TypePal reporta Undefined space `Person`.

Tabla de contenido

- 1. Objetivo de la version actualizada
- 2. Cumplimiento del enunciado
- 3. Matriz de rubrica 15/15
- 4. Cambios implementados para corregir los descuentos
- 5. Gramatica concreta: tipos de usuario y estructuras tipadas
- 6. AST y generador actualizados
- 7. Checker con TypePal: correspondencia de tipos y anidamientos
- 8. Regla nueva de existencia de elementos/tipos en estructuras
- 9. Flujo Main.rsc y salida por consola
- 10. Archivos de prueba y salidas esperadas
- 11. Como probar el proyecto
- 12. Estructura final del proyecto
- 13. Fuente de la solucion
- 14. Conclusiones

1. Objetivo de la version actualizada

El objetivo de esta version es dejar explicita la cobertura completa de la rubrica del Proyecto 3. La entrega reutiliza la definicion previa de VeriLang en Rascal y la completa con parser, sintaxis abstracta, generador, integracion de TypePal, anotaciones de tipos, chequeo semantico, regla de existencia para estructuras y un conjunto de pruebas validas e invalidas.

La mejora principal frente a la version anterior es que las estructuras definidas por el usuario ya no son solo nombres de espacios. Ahora pueden declarar el tipo de elemento que contienen. Por ejemplo:

```

defspace Person end
defspace Group : Person end

```

Con esta declaracion, el sistema entiende que Group es una estructura cuyos elementos son Person. Por tanto, en un cuantificador como forall member in Group, la variable member se tipa como Person y no como Group.

2. Cumplimiento del enunciado

#	Requisito del enunciado	Evidencia en la solucion	Estado
1	Definir el parser del lenguaje	Parser.rsc expone parseMainModule(str, loc) y parseMainModule(loc), usando la gramatica concreta de Syntax.rsc.	Cumplido
2	Generar codigo desde sintaxis y AST usando entrada .vl y mostrar resultado en consola	Main.rsc recibe un loc de archivo .vl, llama a parseMainModule, ejecuta checker, invoca Generator.rsc y muestra el resultado generado en consola.	Cumplido
3	Instalar TypePal en el lenguaje	Checker.rsc extiende analysis::typepal::TypePal. El proyecto declara la libreria typepal en RASCAL.MF / configuracion del proyecto.	Cumplido
4	Actualizar la gramatica con anotaciones para valores y estructuras con tipo dado por usuario	Domain incluye tipos primitivos y nameDomain: ID. Space ahora incluye typedSpace: defspace ID : Domain end y typedSubspace.	Cumplido 5/5
5	Chequear que los tipos usados correspondan a sus definiciones	Checker.rsc valida variables, relaciones, operadores logicos, operadores infijos personalizados, defrule, invocaciones y cuantificadores sobre estructuras tipadas.	Cumplido 5/5
6	Definir una regla nueva para asegurar que los elementos usados en una estructura existan	Todo Domain de usuario usado en variables, operadores y estructuras tipadas se valida con c.use(..., {spaceId()}). Si no existe, se reporta Undefined space.	Cumplido 5/5
Doc.	Documento explicativo	Este documento describe cambios, archivos, pruebas, salidas esperadas y como ejecutar el proyecto.	Cumplido

3. Matriz de rubrica 15/15

3.1 Grammar Update & Type Annotations

Aspecto	Implementacion actual	Por que corresponde a Excellent
Tipos primitivos	Domain soporta int, bool, real, string y char.	Cubre las anotaciones solicitadas para valores del lenguaje.
Tipos de usuario	Domain soporta nameDomain: ID domainName.	Permite declarar variables y operadores con tipos definidos por el usuario, como Person y Group.
Estructuras tipadas	Space soporta defspace Group : Person end.	La estructura de datos ya incluye un tipo dado por el usuario, que era el punto faltante en la retroalimentacion.
Consistencia AST/generator	AST.rsc y Generator.rsc incluyen typedSpace y typedSubspace.	El parser no solo acepta la sintaxis; el resto del lenguaje tambien la conserva y la genera.

3.2 Type Correspondence Checking

Aspecto	Implementacion actual	Evidencia
Variables	Cada defvar se registra con el tipo declarado.	p : Person queda como userType("Person").

Operadores infijos	El operador personalizado debe tener firma completa A -> B -> C.	belongsTo : Person -> Group -> bool valida lado izquierdo, lado derecho y retorno.
Cuantificadores sobre estructuras	forall member in Group asigna member : Person si Group : Person.	El checker usa typedSpaceElementTypes y quantifiedVariableType.
Casos anidados	El cuerpo del cuantificador se chequea dentro de un scope propio y debe retornar bool.	forall member in Group . member belongsTo g pasa porque member es Person y g es Group.
Errores de tipos	Si el operador espera Set pero recibe Element, se reporta error.	spec_typed_space_quantifier_type_error.vl falla con "El lado izquierdo del operador no coincide con su firma".

3.3 Data Structure Element Existence Rule

Aspecto	Implementacion actual	Evidencia
Tipo de elemento de una estructura	defspace Group : Person end obliga a que Person exista.	Si falta defspace Person end, TypePal reporta Undefined space `Person`.
Tipos de usuario en variables	x : Group obliga a que Group exista.	Se valida mediante c.use sobre Domain de usuario.
Tipos de usuario en operadores	defoperator belongsTo : Person -> Group -> bool exige que Person y Group existan.	useDomainIfUserDefined se aplica a cada Domain de la firma.
Casos validos	defspace Person end seguido de defspace Group : Person end pasa.	spec_types.vl y spec_typed_space_quantifier_good.vl producen list[Message]: [].

4. Cambios implementados para corregir los descuentos

Archivo	Cambio realizado	Criterio de rubrica que fortalece
Syntax.rsc	Se agrego typedSpace y typedSubspace para permitir defspace Nombre : Tipo end.	Grammar Update & Type Annotations
AST.rsc	Se agregaron los constructores typedSpace(str, Domain) y typedSubspace(str, str, Domain).	Grammar Update & Type Annotations
Generator.rsc	Se agrego generacion textual para defspace Group : Person end.	Grammar Update & Type Annotations / generador
Checker.rsc	Se agrego typedSpaceElementTypes, registerTypedSpaces y quantifiedVariableType.	Type Correspondence Checking
Checker.rsc	Se removio el bypass que permitia tratar operadores booleanos sin validar operandos.	Type Correspondence Checking
Checker.rsc	Se agregaron collect para nodos contenedores y se limpiaron mensajes info innecesarios.	Calidad de entrega y evidencia limpia
Main.rsc	Se ajusto el flujo para ejecutar spec_types.vl por defecto, validar errores y mostrar resultado en consola.	Parser + TypePal + generador
instance/spec_types.vl	Se actualizo para incluir defspace Group : Person end y forall member in Group.	Evidencia directa para rubrica
instance/errors/spec_unknown_space_error.vl	Se actualizo para demostrar error en estructura tipada con tipo inexistente.	Data Structure Element Existence Rule
instance/errors/spec_typed_space_quantifier_type_error.vl	Se agrego caso donde el elemento de estructura tiene tipo correcto y detecta operador incorrecto.	Type Correspondence Checking en anidamientos

5. Gramatica concreta: tipos de usuario y estructuras tipadas

La regla Domain representa tanto los tipos primitivos como los tipos definidos por el usuario. Los tipos definidos por usuario se escriben como ID y deben existir como defspace cuando se usan.

```
syntax Domain
= boolDomain: 'bool'
| intDomain: 'int'
| realDomain: 'real'
| stringDomain: 'string'
| charDomain: 'char'
```

```
| nameDomain: ID domainName
;
```

La regla Space fue extendida para permitir estructuras tipadas. Este es el cambio central para cumplir el nivel Excellent del criterio Grammar Update & Type Annotations.

```
syntax Space
= space: 'defspace' ID spaceName 'end'
| typedSpace: 'defspace' ID spaceName ':' Domain elementType 'end'
| subspace: 'defspace' ID subSpace '\<' ID superSpace 'end'
| typedSubspace: 'defspace' ID subSpace '\<' ID superSpace ':' Domain elementType 'end'
;
```

6. AST y generador actualizados

6.1 AST.rsc

El AST conserva el tipo de elemento de cada estructura tipada. Sin este cambio, el parser podria aceptar la sintaxis, pero el checker y el generador perderian la informacion semantica.

```
data Space
= space(str spaceName)
| typedSpace(str spaceName, Domain elementType)
| subspace(str subSpace, str superSpace)
| typedSubspace(str subSpace, str superSpace, Domain elementType)
;
```

6.2 Generator.rsc

El generador tambien fue actualizado para emitir nuevamente la sintaxis de estructuras tipadas. Esto demuestra que la nueva informacion no solo se parsea, sino que se conserva durante todo el flujo.

```
str generate(typedSpace(str spaceName, Domain elementType)) {
  return "defspace <spaceName> : <generate(elementType)> end";
}

str generate(typedSubspace(str sub, str super, Domain elementType)) {
  return "defspace <sub> \< <super> : <generate(elementType)> end";
}
```

7. Checker con TypePal: correspondencia de tipos y anidamientos

Checker.rsc define los tipos semanticos del lenguaje. Los tipos de usuario se representan como userType(str name), y las firmas de operadores se representan con functionType(list[AType]).

```
data AType
= boolType()
| intType()
| realType()
| stringType()
| charType()
| userType(str name)
| functionType(list[AType] signature)
;
```

7.1 Registro del tipo de elemento de cada estructura

Para resolver el problema de definiciones anidadas, el checker registra que estructuras como Group contienen elementos de tipo Person. Asi, en forall member in Group, member no queda tipado como Group sino como Person.

```
map[str, AType] typedSpaceElementTypes = ();

void registerTypedSpaces(Tree pt) {
  typedSpaceElementTypes = ();

  if (pt has top) {
    pt = pt.top;
  }
}
```

```

visit(pt) {
  case (Space) `defspace <ID name> : <Domain elementType> end` : {
    typedSpaceElementTypes["<name>"] = domainToType(elementType);
  }

  case (Space) `defspace <ID sub> \< <ID super> : <Domain elementType> end` : {
    typedSpaceElementTypes["<sub>"] = domainToType(elementType);
  }
}
}

```

7.2 Tipo correcto dentro de cuantificadores

```

AType quantifiedVariableType(str domainName) {
  if (domainName in typedSpaceElementTypes) {
    return typedSpaceElementTypes[domainName];
  }

  return userType(domainName);
}

```

Con esto, el siguiente programa valido pasa sin mensajes de error:

```

defspace Person end
defspace Group : Person end

defvar
  g : Group
end

defoperator belongsTo : Person -> Group -> bool end

defexpression (forall member in Group . member belongsTo g) end

```

7.3 Validacion estricta de operadores infijos

Los operadores infijos personalizados ya no se aceptan automaticamente por ser booleanos. El checker exige una firma completa y compara el tipo real de cada operando con la firma declarada.

```

defoperator belongsTo : Person -> Group -> bool end
defexpression member belongsTo g end

-- member debe ser Person
-- g debe ser Group
-- el resultado es bool

```

8. Regla nueva de existencia de elementos/tipos en estructuras

La regla nueva se interpreta de forma concreta para VeriLang asi: si una estructura declara un tipo de elemento definido por el usuario, ese tipo debe existir como defspace. Tambien se aplica a variables y operadores que usen tipos de usuario.

8.1 Caso valido

```

defmodule ValidStructure

  defspace Person end
  defspace Group : Person end

end

```

8.2 Caso invalido

```

defmodule UnknownSpaceError

  defspace Group : Person end

```

```

end

-- Error esperado:
-- Undefined space `Person`

```

8.3 Implementacion en Checker.rsc

```

void useDomainIfUserDefined(Domain d, Collector c) {
    if (domainToType(d) is userType) {
        c.use(d, {spaceId()});
    }
}

void collect(
    current: (Space) `defspace <ID name> : <Domain elementType> end`,
    Collector c
) {
    c.define("<name>", spaceId(), name, defType(userType("<name>")));
    useDomainIfUserDefined(elementType, c);
}

```

9. Flujo Main.rsc y salida por consola

Main.rsc implementa el flujo completo solicitado: recibe un archivo .vl, lo parsea, ejecuta TypePal, bloquea la generacion si hay errores y, si no hay errores, imprime el codigo generado en consola.

```

import Main;
main();

-- Salida esperada:
-- Parser ejecutado correctamente.
-- Checker ejecutado correctamente.
-- No se encontraron errores semanticos.
-- Resultado generado: ...
-- int: 0

main(|file:///C:/Users/Roni/PROYECTO_RASCAL/instance/errors/spec_unknown_space_error.vl|);

-- Salida esperada:
-- El programa contiene errores semanticos:
-- error("Undefined space `Person`, ...")
-- No se genera codigo porque el programa no paso el chequeo semantico.
-- int: 1

```

10. Archivos de prueba y salidas esperadas

Archivo	Tipo	Proposito	Salida esperada
instance/spec_types.vl	Valido	Prueba tipos primitivos, tipos de usuario, defspace Group : Person y cuantificador sobre Group.	specTypes.messages -> list[Message]: []
instance/spec_typed_space_good.vl	Valido	Prueba minima de estructura tipada: Element y Set : Element.	messages -> list[Message]: []
instance/spec_typed_space_quantifier_good.vl	Valido	Prueba que forall e in Set asigna e : Element.	messages -> list[Message]: []
instance/errors/spec_unknown_space_error.vl	Invalido	Prueba defspace Group : Person cuando Person no existe.	Undefined space `Person`
instance/errors/spec_typed_space_unknown_element_error.vl	Invalido	Prueba defspace Set : Element sin defspace Element.	Undefined space `Element`
instance/errors/spec_typed_space_quantifier_type_error.vl	Invalido	Prueba error de tipo dentro de cuantificador sobre estructura tipada.	El lado izquierdo del operador no coincide con su firma
instance/errors/spec_type_error.vl	Invalido	Prueba relacion con tipos incompatibles.	Los dos lados de la relacion deben tener el mismo tipo
instance/errors/	Invalido	Prueba operadores logicos con	El lado izquierdo/derecho de and

spec_logic_error.vl		tipos incorrectos.	debe tener tipo booleano
instance/errors/ spec_operator_error.vl	Invalido	Prueba operador infijo con argumentos incorrectos.	El lado izquierdo/derecho del operador no coincide con su firma
instance/errors/ spec_rule_error.vl	Invalido	Prueba argumentos incorrectos en defrule.	Argumento no coincide con tipo esperado por operador

10.1 Evidencia de prueba valida principal

```
TModel specTypes = modulesTModelFromTree(
    parseMainModule(|file:///C:/Users/Roni/PROYECTO_RASCAL/instance/spec_types.vl|)
);
specTypes.messages;

-- Resultado obtenido:
-- list[Message]: []
```

10.2 Evidencia de prueba invalida de existencia

```
TModel unknownSpace = modulesTModelFromTree(
    parseMainModule(|file:///C:/Users/Roni/PROYECTO_RASCAL/instance/errors/spec_unknown_space_error.vl|)
);
unknownSpace.messages;

-- Resultado obtenido:
-- list[Message]: [error("Undefined space `Person`, ...)]
```

10.3 Evidencia de prueba invalida de anidamiento

```
TModel bad2 = modulesTModelFromTree(
    parseMainModule(|file:///C:/Users/Roni/PROYECTO_RASCAL/instance/errors/
spec_typed_space_quantifier_type_error.vl|)
);
bad2.messages;

-- Resultado obtenido:
-- list[Message]: [error("El lado izquierdo del operador no coincide con su firma", ...)]
```

11. Como probar el proyecto

11.1 Importar modulos principales

```
import Syntax;
import AST;
import Parser;
import Generator;
import Checker;
import Main;
```

11.2 Probar parser

```
parseMainModule(|file:///C:/Users/Roni/PROYECTO_RASCAL/instance/spec_types.vl|);
```

11.3 Probar checker en caso valido

```
TModel specTypes = modulesTModelFromTree(
    parseMainModule(|file:///C:/Users/Roni/PROYECTO_RASCAL/instance/spec_types.vl|)
);
specTypes.messages;

-- Esperado:
-- list[Message]: []
```

11.4 Probar checker en caso invalido

```
TModel unknownSpace = moduleSTModelFromTree(
    parseMainModule(|file:///C:/Users/Roni/PROYECTO_RASCAL/instance/errors/spec_unknown_space_error.vl|)
);
unknownSpace.messages;

-- Esperado:
-- Undefined space `Person`
```

11.5 Probar flujo completo

```
import Main;
main();

-- Esperado para caso valido:
-- int: 0

main(|file:///C:/Users/Roni/PROYECTO_RASCAL/instance/errors/spec_unknown_space_error.vl|);

-- Esperado para caso invalido:
-- int: 1
```

12. Estructura final del proyecto

```
PROYECTO_RASCAL/
  META-INF/
  RASCAL.MF
  src/main/rascal/
    AST.rsc
    Checker.rsc
    Generator.rsc
    Implode.rsc
    Main.rsc
    Parser.rsc
    Plugin.rsc
    Syntax.rsc
  instance/
    spec1.vl
    spec2.vl
    spec3.vl
    spec_types.vl
    spec_typed_space_good.vl
    spec_typed_space_quantifier_good.vl
    errors/
      spec_type_error.vl
      spec_unknown_space_error.vl
      spec_logic_error.vl
      spec_operator_error.vl
      spec_rule_error.vl
      spec_typed_space_unknown_element_error.vl
      spec_typed_space_quantifier_type_error.vl
    output/
      testGenerator.vl
      testGenerator2.vl
  pom.xml
```

13. Fuente de la solucion

La solucion parte del proyecto propio desarrollado en las iteraciones anteriores de VeriLang. No se tomo una solucion externa completa como base. Los cambios finales se implementaron directamente sobre los archivos del proyecto propio, principalmente Syntax.rsc, AST.rsc, Generator.rsc, Checker.rsc, Main.rsc y los programas de prueba en instance/.

14. Conclusiones

La version actualizada de VeriLang cubre los requisitos centrales del Proyecto 3 y fortalece explicitamente los puntos de la rubrica. La gramatica ya incluye estructuras con tipo de elemento definido por el usuario; el checker valida esos tipos dentro de expresiones anidadas y cuantificadores; y la regla nueva de existencia detecta tipos de usuario inexistentes en estructuras, variables y operadores.

La evidencia principal es que los casos validos producen list[Message]: [] y los casos invalidos producen errores semanticos concretos. Ademas, Main.rsc ejecuta el flujo completo sobre archivos .vl, muestra el resultado generado en consola y bloquea la generacion cuando existen errores semanticos. Con estas correcciones, la entrega queda alineada con el nivel Excellent de los tres criterios evaluados en la rubrica.